Sesame Workshop — Data & Analytics Take-Home

Welcome! This exercise mirrors the actual first month of this job. Plan for **3–4 hours**. We are not timing you, but we calibrate everything to that budget — a correct, well-reasoned subset beats a rushed everything, and we read volume beyond the budget as a miss, not as diligence.

Scenario

You are the first dedicated analytics engineer at **Sesame Workshop**. The digital team instruments our web property with **Google Analytics**; Development (fundraising) lives in **Salesforce**; Content Distribution receives weekly **TV-ratings deliveries** from our measurement vendor. Three raw feeds land in a bucket. Nobody has ever modeled them. Your job is not to analyze this data — it is to build the **modeled layer** that makes analysis cheap, correct, and repeatable for the analysts who come after you, and to make the judgment calls that layer depends on.

The data (`data/`)

Path	What it is
`events/events_YYYYMMDD.jsonl`	Daily Google Analytics (GA4) BigQuery-style export: nested `event_params`, device-scoped `user_pseudo_id`, timestamps in UTC microseconds. 14 days.
`events/events_intraday_20260714.jsonl`	The intraday export for the last day. Our loader copies whatever is in the bucket.
`salesforce/contacts.csv`, `salesforce/donations.csv`, `salesforce/campaigns.csv`	Nightly Salesforce extract. `donations.transaction_id` is populated for web-originated gifts.
`ratings/ratings_weekly_YYYYMMDD.csv`	Weekly TV-ratings deliveries; append-only, the filename is the delivery date. Each telecast first reports a Live+Same-Day figure (`stream = live_sd`); a restated Live+7 figure (`live_7`) arrives in a later delivery once the seven-day viewing window has closed. Nothing in a file marks corrections or restatements.

The feeds have the problems real feeds have. Handling the ones that change answers is the exercise; log the ones you notice but don't fix.

What to build

One entrypoint rebuilds everything from the raw files and produces the three verification answers: `uv run python run_pipeline.py`. Use **DuckDB SQL and/or polars** (see `pyproject.toml`). A full rebuild on every run is fine.

There is no required model list and no required test framework. **Build only the modeled layer you need to answer the questions and stand behind your memo. What you choose *not* to build is part of the assessment.**

Verification questions

Deterministic under these pinned definitions; produce them from your models (`answers.md` or printed by the pipeline):

- **Q1.** A session is `(user_pseudo_id, ga_session_id)`; events without `ga_session_id` belong to no session. How many sessions have their first event between `2026-07-01 00:00:00` and `2026-07-07 23:59:59` **UTC** (inclusive week), and what is the median session duration (last event – first event; single-event sessions count as 0) for those sessions, in seconds?
- *Memo prompt for Q1:* GA4's `event_date` is property-local time. State the time semantics your models treat as canonical, and what that means for when a day's partition is closed.

- **Q2.** Across the full 14 days: how many **distinct** web transactions (`donation_complete.transaction_id`) are there, how many match a Salesforce donation, and what is the attribution rate?
- *Memo prompt for Q2:* what do you tell the fundraising team about the transactions that don't match — and does a donation dated outside its campaign's window count toward that campaign's total?
- **Q3.** A telecast's reportable audience is its `live_7` row if one has been delivered, otherwise its `live_sd` row; if the same telecast and stream appear in more than one delivery file, the most recent delivery wins. For **Sesame Street** telecasts with `air_date` in the broadcast week Mon `2026-07-06` through Sun `2026-07-12`: **(a)** the total reportable audience (sum of `avg_audience_k`) using every delivered file; **(b)** the same total as it would have been reported using only files delivered on or before `2026-07-21`; **(c)** the count of site `video_start` events for Sesame Street episodes (`episode_id` beginning `SST-`) in that same week, UTC.
- *Memo prompt for Q3:* which of (a)/(b) goes in the board deck, what is your rule for calling a broadcast week final, and what caveat goes under a slide that shows a panel-based audience estimate next to a device-scoped event count?

DECISION_MEMO.md — max 1 page

The memo is the primary artifact; we grade the decisions as heavily as the pipeline. For each decision: your choice, the alternative you rejected, and what changes downstream if it flips.

- The three memo prompts above (time semantics, unmatched and out-of-window donations, ratings reporting policy).
- **Identity:** if fundraising asks "which donors watched a video before giving," what does the device↔contact mapping in this data honestly support? State the rule you would use and what you would refuse to claim.
- **Refused / deferred:** what you deliberately did not build or fix, and why.
- **Incremental loading (design only — do not implement):** if this ran hourly: merge key, watermark, late-data policy, intraday-file policy.
- 2–3 questions for the upstream system owners.
- **AI-use disclosure:** how you used AI tools, and at least one place where you checked, corrected, or overrode what they produced.

Ground rules

- **AI tools are allowed and expected.** We grade the decisions and we will work through your submission together in a live follow-up — including modifying it. Make sure every decision in DECISION_MEMO.md is genuinely yours.
- Everything runs locally from the raw files. No cloud warehouse, no network calls.
- Submit a zip or repo link: code, DECISION_MEMO.md, and a README with how to run it and roughly how you spent your time.